ARIMA Analysis of Kalshi Prediction Market Prices

Load and Prepare Data

```
trades <- read_csv("trades_clean.csv")

# Parse timestamps, sort by contract then time
trades <- trades %>%
  mutate(created_time = as.POSIXct(created_time, tz = "UTC")) %>%
  arrange(ticker, created_time)

glimpse(trades)
```

```
## Rows: 106,955
## Columns: 12
## $ trade_id        <chr> "c4c7a70c-2a31-5f87-63e5-d8343cc88aca", "9bc73c2f-79bc…
## $ ticker          <chr> "KXALBUMRELEASEDATELUCKI-APR01-26", "KXALBUMRELEASEDAT…
## $ count           <dbl> 50, 5, 200, 49, 32, 93, 100, 5, 50, 10, 1, 1, 32, 28, …
## $ yes_price       <dbl> 0.96, 0.62, 0.24, 0.97, 0.22, 0.21, 0.05, 0.06, 0.02, …
## $ no_price        <dbl> 0.04, 0.38, 0.76, 0.03, 0.78, 0.79, 0.95, 0.94, 0.98, …
## $ taker_side      <chr> "yes", "yes", "yes", "yes", "no", "no", "no", "no", "n…
## $ created_time    <dttm> 2026-02-03 16:58:19, 2026-02-03 16:58:19, 2026-02-03 …
## $ `_fetched_at`   <dttm> 2026-04-03 14:12:31, 2026-04-03 14:12:31, 2026-04-03 …
## $ result          <chr> "no", "no", "no", "no", "no", "no", "no", "no", "no", …
## $ close_time      <dttm> 2026-04-01 14:00:00, 2026-04-01 14:00:00, 2026-04-01 …
## $ hours_to_expiry <dbl> 1365.0278, 1365.0278, 1365.0278, 1365.0278, 1308.6436,…
## $ outcome         <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, …
```

Pick a Representative Contract

Select a contract with enough trades to fit a time series model, use `yes_price` as the price series.

```
# Find contracts with the most trades
trade_counts <- trades %>%
  count(ticker, sort = TRUE)

print(head(trade_counts, 10))
```

```
## # A tibble: 10 × 2
##    ticker                                     n
##    <chr>                                  <int>
##  1 KXUSAIRANAGREEMENT-27-26APR             8379
##  2 KXATPCHALLENGERMATCH-26APR02GUIHUE-HUE  8355
##  3 KXDHSFUNDING-26APR01                    5280
##  4 KXATPCHALLENGERMATCH-26APR02GUIHUE-GUI  4776
##  5 KXATPCHALLENGERMATCH-26APR02HEIPRA-PRA  3554
##  6 KXNBASPREAD-26APR02SASLAC-SAS19         2928
##  7 KXNBASPREAD-26APR02CLEGSW-CLE10         2730
##  8 KXNBASPREAD-26APR02SASLAC-SAS10         2456
##  9 KXNBASPREAD-26APR02SASLAC-SAS4          2181
## 10 KXATPCHALLENGERMATCH-26APR02HEIPRA-HEI  2096
```

```
# Use the most-traded contract
selected_ticker <- trade_counts$ticker[1]
cat("Selected ticker:", selected_ticker, "\n")
```

```
## Selected ticker: KXUSAIRANAGREEMENT-27-26APR
```

```
contract_trades <- trades %>%
  filter(ticker == selected_ticker) %>%
  select(created_time, yes_price, outcome) %>%
  arrange(created_time)

cat("Number of trades:", nrow(contract_trades), "\n")
```

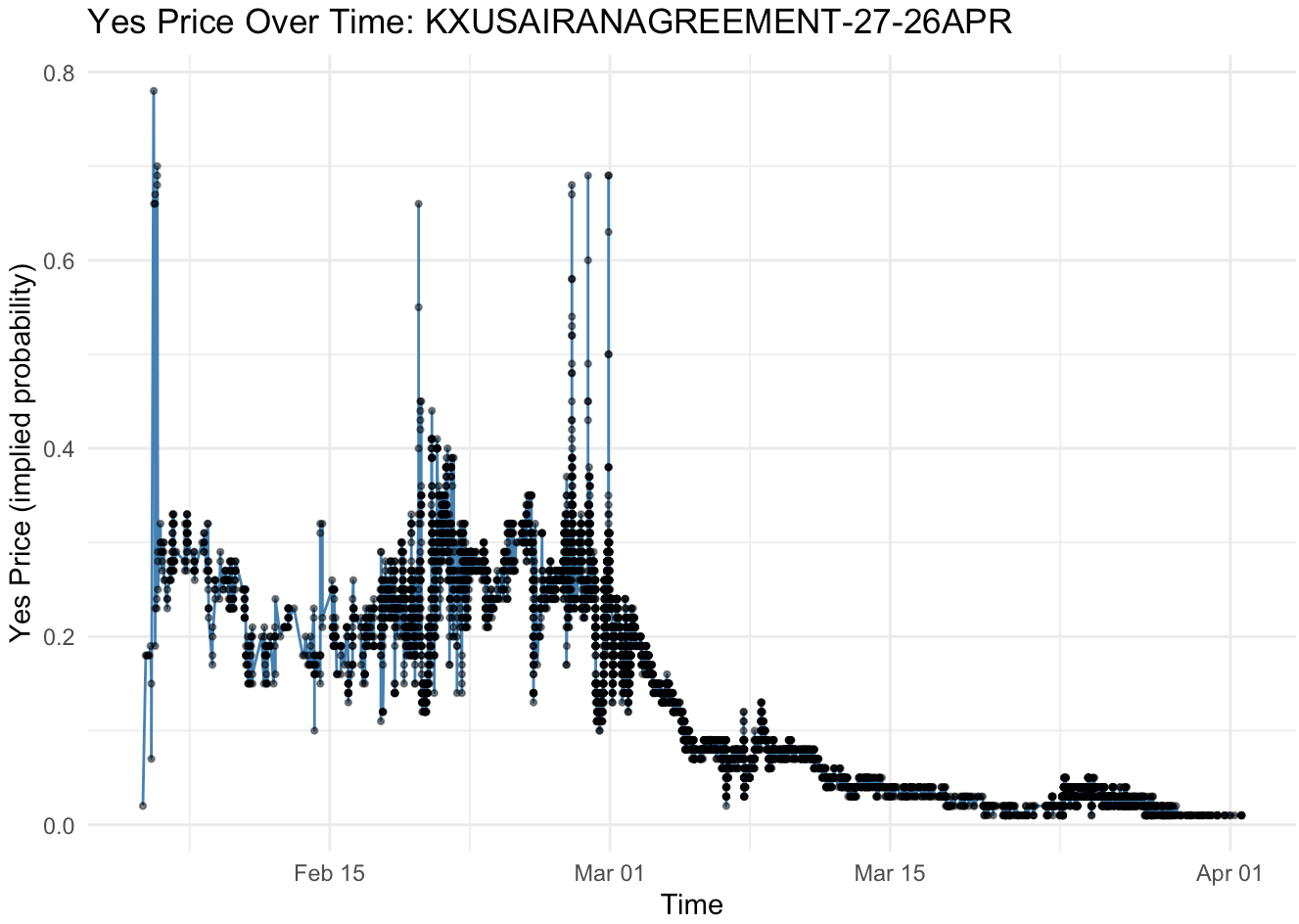
```
## Number of trades: 8379
```

```
cat("Outcome:", unique(contract_trades$outcome), "\n")
```

```
## Outcome: 0
```

Price Series Plot

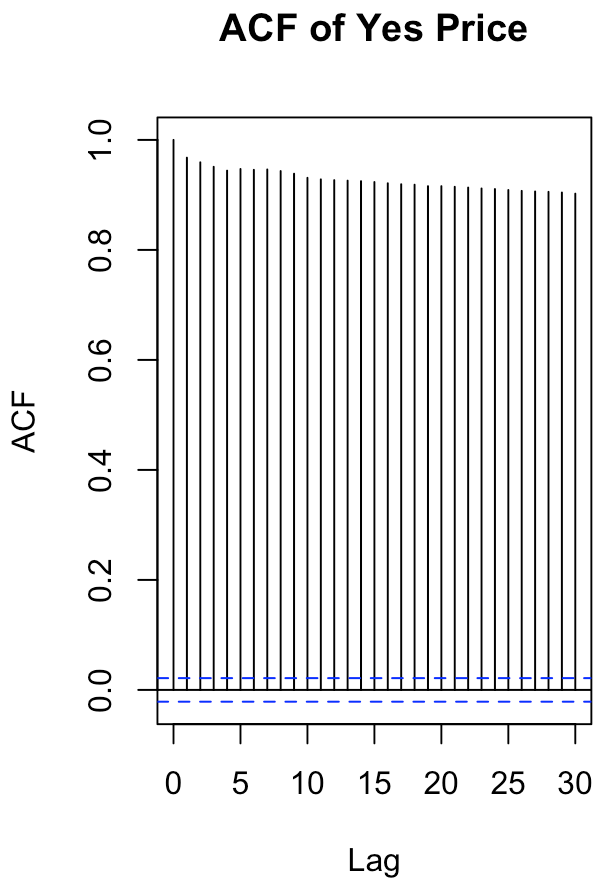
```
ggplot(contract_trades, aes(x = created_time, y = yes_price)) +
  geom_line(color = "steelblue") +
  geom_point(size = 0.8, alpha = 0.5) +
  labs(
    title = paste("Yes Price Over Time:", selected_ticker),
    x = "Time",
    y = "Yes Price (implied probability)"
  ) +
  theme_minimal()
```

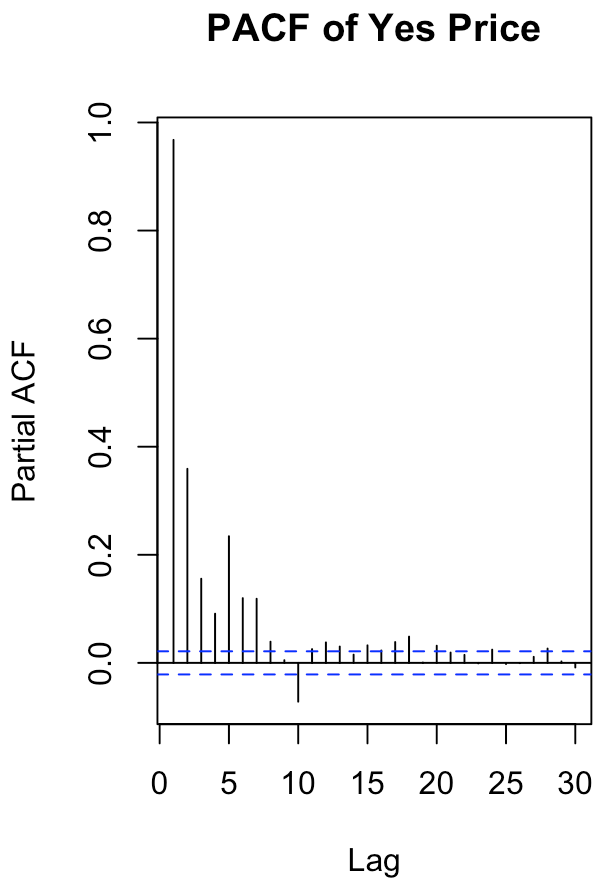


ACF and PACF of Raw Price Series

Before fitting ARIMA, inspect autocorrelation in the raw price series.

```
price_ts <- ts(contract_trades$yes_price)

par(mfrow = c(1, 2))
acf(price_ts, main = "ACF of Yes Price", lag.max = 30)
pacf(price_ts, main = "PACF of Yes Price", lag.max = 30)
```






```
par(mfrow = c(1, 1))
```

Stationarity Test (ADF)

An Augmented Dickey-Fuller test checks for a unit root, a hallmark of a random walk.

- **Null hypothesis**: series has a unit root (random walk / non-stationary)
- **p < 0.05**: reject the null → series is stationary

```
adf_result <- adf.test(price_ts)
print(adf_result)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  price_ts
## Dickey-Fuller = -5.2591, Lag order = 20, p-value = 0.01
## alternative hypothesis: stationary
```

Fit ARIMA Model

Use `auto.arima()` to select the best ARIMA(p, d, q) model based on AIC. A model with `d = 1` indicates the series needed differencing, which is consistent with a random walk.

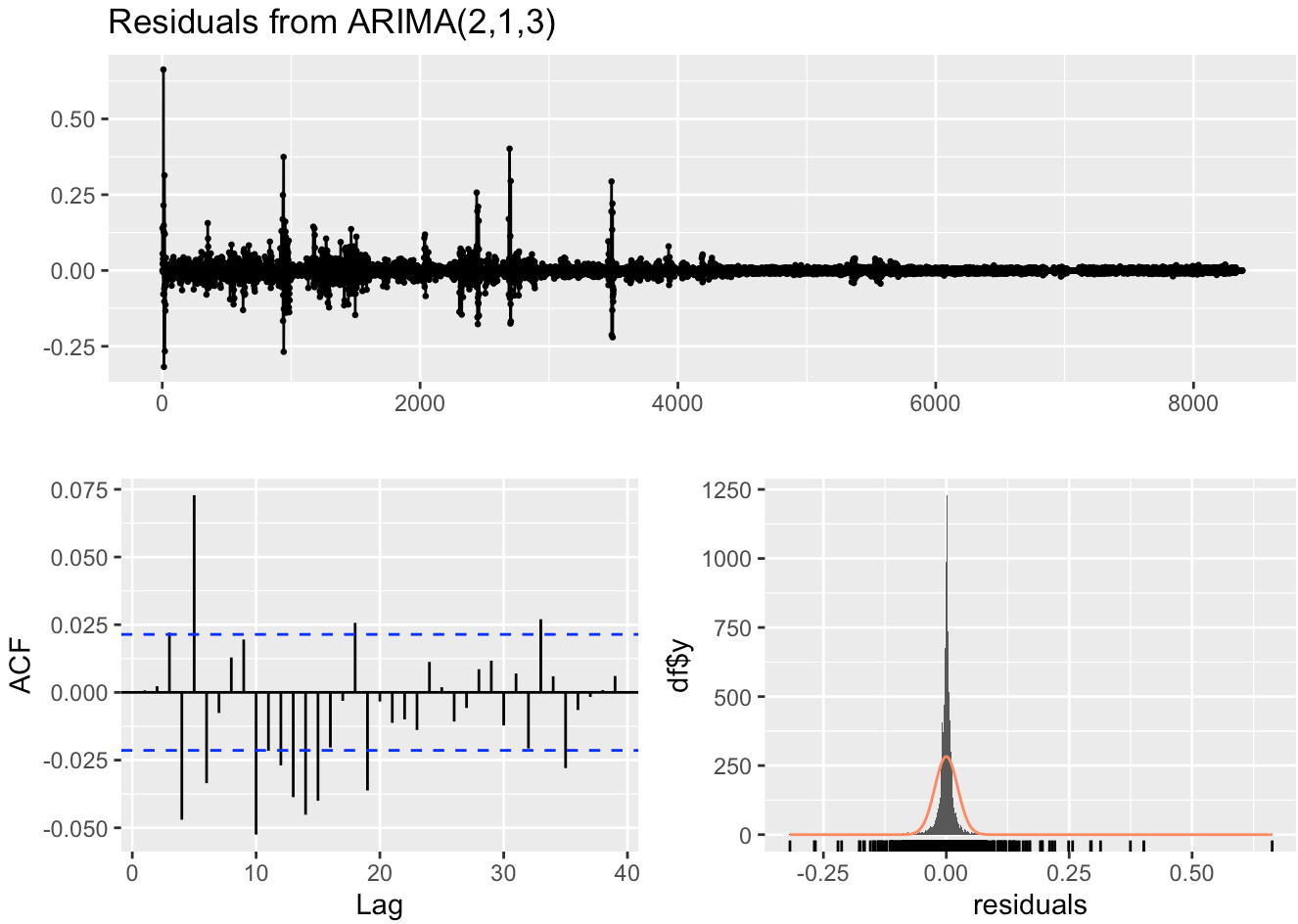
```
arima_model <- auto.arima(price_ts, stepwise = FALSE, approximation = FALSE)
summary(arima_model)
```

```
## Series: price_ts
## ARIMA(2,1,3)
##
## Coefficients:
##          ar1      ar2      ma1     ma2      ma3
##       1.1288  -0.8126  -1.6560  1.3632  -0.4772
## s.e.  0.0334   0.0248   0.0336  0.0409   0.0165
##
## sigma^2 = 0.0005385:  log likelihood = 19643.92
## AIC=-39275.85   AICc=-39275.84   BIC=-39233.65
##
## Training set error measures:
##                        ME       RMSE       MAE       MPE     MAPE     MASE
## Training set -2.160986e-05 0.02319677 0.0101374 -1.671223 8.356032 1.048144
##                      ACF1
## Training set 0.0007628396
```

Residual Diagnostics

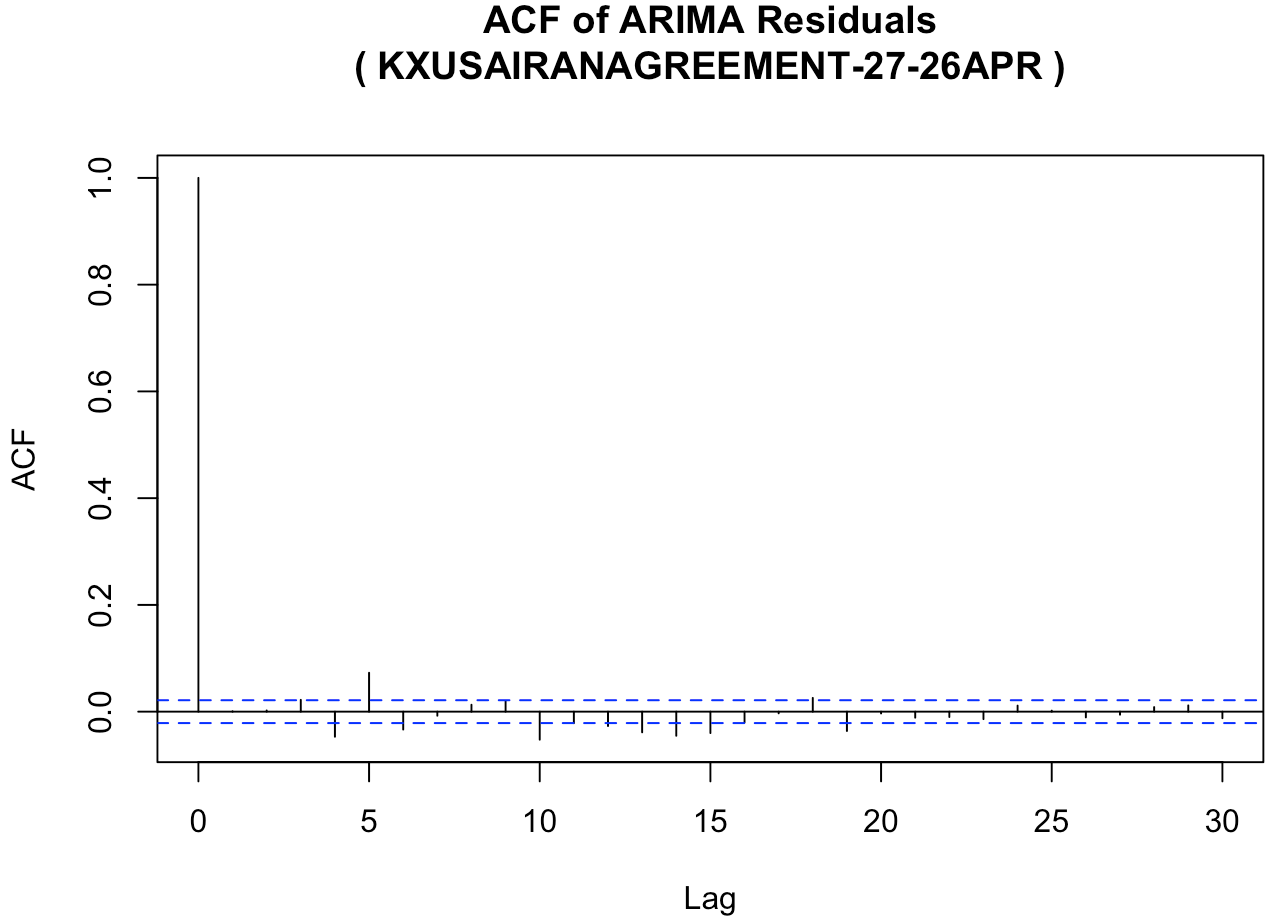
If prices are a random walk, ARIMA residuals should be white noise with no autocorrelation. If residuals show autocorrelation, past prices carry predictive information.

```
checkresiduals(arima_model)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(2,1,3)
## Q* = 104.81, df = 5, p-value < 2.2e-16
##
## Model df: 5.   Total lags used: 10
```

```
acf(residuals(arima_model),
    main = paste("ACF of ARIMA Residuals\n(", selected_ticker, ")"),
    lag.max = 30)
```



Ljung-Box Test on Residuals

Formally tests whether residuals are white noise across multiple lags.

- **Null hypothesis**: residuals are white noise (random walk behavior)
- **p < 0.05**: significant autocorrelation remains → past prices are predictive

```
lb_test <- Box.test(residuals(arima_model), lag = 20, type = "Ljung-Box")
print(lb_test)
```

```
##
##  Box-Ljung test
##
## data:  residuals(arima_model)
## X-squared = 178.18, df = 20, p-value < 2.2e-16
```

Interpretation

```
d_order <- arima_model$arma[6]  # differencing order from auto.arima
lb_p <- lb_test$p.value
adf_p <- adf_result$p.value

cat("ARIMA model order: ARIMA(",
    arima_model$arma[1], ",",
    arima_model$arma[6], ",",
    arima_model$arma[2], ")\n")
```

```
## ARIMA model order: ARIMA( 2 , 1 , 3 )
```

```
cat("ADF p-value:", round(adf_p, 4), "\n")
```

```
## ADF p-value: 0.01
```

```
cat("Ljung-Box p-value:", round(lb_p, 4), "\n\n")
```

```
## Ljung-Box p-value: 0
```

```
if (d_order >= 1 && lb_p > 0.05) {
  cat("Conclusion: Price series requires differencing (d >= 1) and residuals show\n",
      "no significant autocorrelation. This is consistent with a RANDOM WALK —\n",
      "past prices do not predict future prices beyond what is already priced in.\n")
} else if (lb_p <= 0.05) {
  cat("Conclusion: Significant autocorrelation remains in residuals (Ljung-Box p =",
      round(lb_p, 4), ").\n",
      "Past prices carry predictive information — the series is NOT a pure random wal
k.\n",
      "This suggests price movements are autocorrelated, potentially reflecting\n",
      "the market updating on new information in a non-random pattern.\n")
} else {
  cat("Conclusion: Mixed evidence. Inspect ACF plot and model order for further insigh
t.\n")
}
```

```
## Conclusion: Significant autocorrelation remains in residuals (Ljung-Box p = 0 ).
##  Past prices carry predictive information — the series is NOT a pure random walk.
##  This suggests price movements are autocorrelated, potentially reflecting
##  the market updating on new information in a non-random pattern.
```

Repeat Across Multiple Contracts

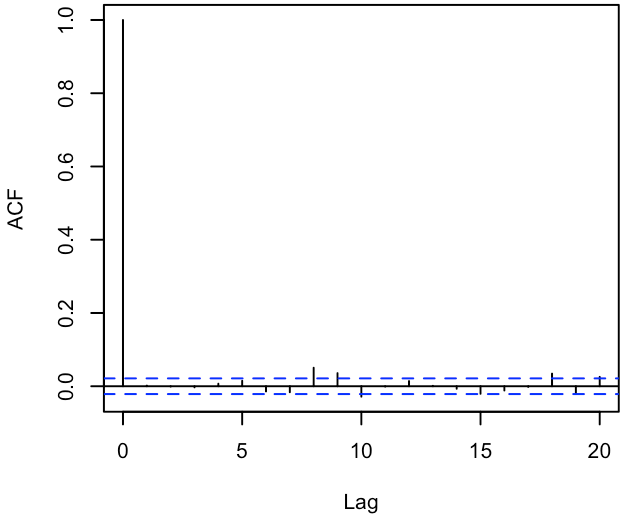
Run the same analysis across several contracts to see whether the random walk result generalizes.

```
# Pick top 5 most-traded contracts
top_tickers <- trade_counts$ticker[1:5]

results <- map_dfr(top_tickers, function(tkr) {
  df <- trades %>%
    filter(ticker == tkr) %>%
    arrange(created_time)

  if (nrow(df) < 20) return(NULL)

  ts_data <- ts(df$yes_price)

  tryCatch({
    model <- auto.arima(ts_data, stepwise = TRUE, approximation = TRUE)
    lb <- Box.test(residuals(model), lag = 10, type = "Ljung-Box")
    adf <- adf.test(ts_data)

    tibble(
      ticker = tkr,
      n_trades = nrow(df),
      outcome = unique(df$outcome)[1],
      arima_order = paste0("(", model$arma[1], ",", model$arma[6], ",", model$arma[2],
")"),
      d_order = model$arma[6],
      adf_p = round(adf$p.value, 4),
      ljung_box_p = round(lb$p.value, 4),
      random_walk = ifelse(model$arma[6] >= 1 & lb$p.value > 0.05, "Yes", "No")
    )
  }, error = function(e) NULL)
})

print(results)
```

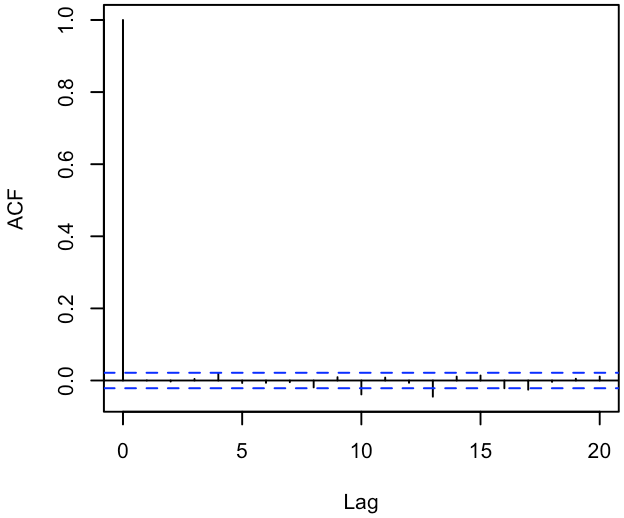
```
## # A tibble: 5 × 8
##   ticker      n_trades outcome arima_order d_order adf_p ljung_box_p random_walk
##   <chr>          <int>   <dbl> <chr>         <int> <dbl>       <dbl> <chr>
## 1 KXUSAIRANA…     8379       0 (5,1,5)           1 0.01       0      No
## 2 KXATPCHALL…     8355       1 (2,1,2)           1 0.276      0.027  No
## 3 KXDHSFUNDI…     5280       0 (1,1,2)           1 0.582      0.0059 No
## 4 KXATPCHALL…     4776       0 (5,1,1)           1 0.560      0.0009 No
## 5 KXATPCHALL…     3554       0 (5,1,2)           1 0.282      0.313  Yes
```

```
# ACF plots for top contracts
par(mfrow = c(ceiling(length(top_tickers) / 2), 2))

for (tkr in top_tickers) {
  df <- trades %>%
    filter(ticker == tkr) %>%
    arrange(created_time)

  if (nrow(df) < 20) next

  tryCatch({
    model <- auto.arima(ts(df$yes_price), stepwise = TRUE, approximation = TRUE)
    acf(residuals(model),
        main = paste0(substr(tkr, 1, 30), "\nARIMA", paste0("(", model$arma[1], ",", mod
el$arma[6], ",", model$arma[2], ")")),
        lag.max = 20)
  }, error = function(e) NULL)
}

par(mfrow = c(1, 1))
```

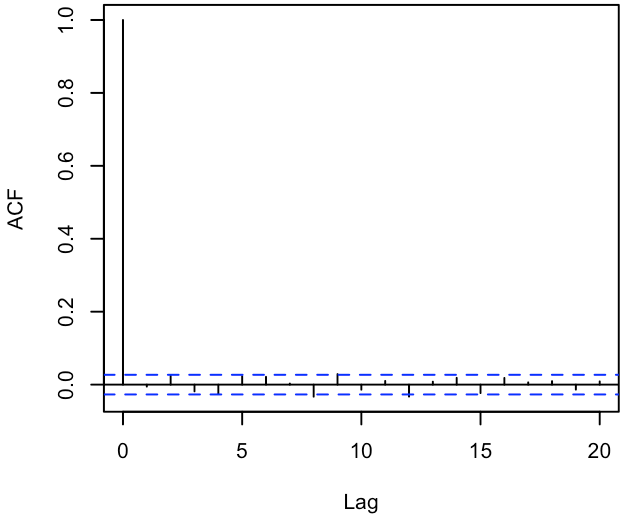
KXUSAIRANAGREEMENT-27-26APR
ARIMA(5,1,5)



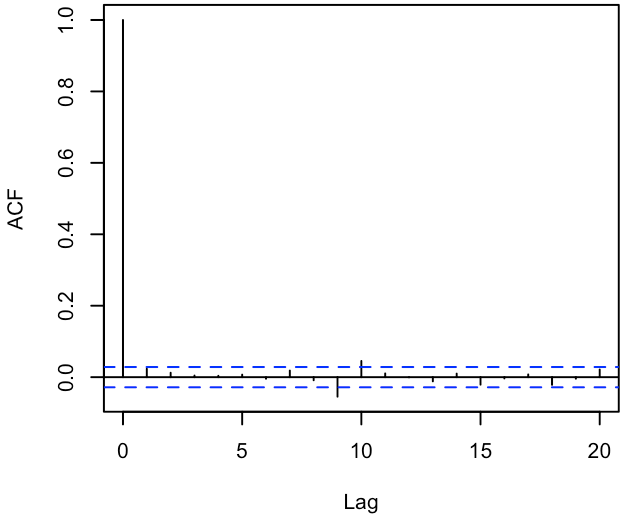
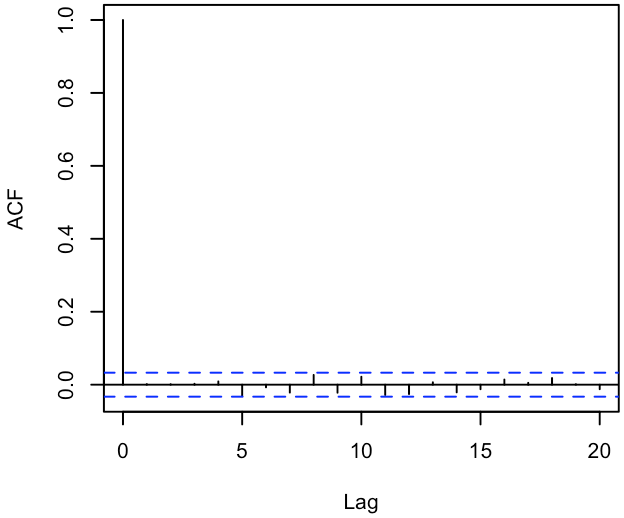
KXATPCHALLENGERMATCH-26APR02GU
ARIMA(2,1,2)



KXDHSFUNDING-26APR01
ARIMA(1,1,2)



KXATPCHALLENGERMATCH-26APR02GU
ARIMA(5,1,1)



KXATPCHALLENGERMATCH-26APR02HE
ARIMA(5,1,2)